EST-744

Clasificación de texto con redes neuronales

Juan Zamora O.

Deep Learning





El Problema con el Promediado

Promediar embeddings ignora la **estructura de secuencias**:

```
"No me gustó esta película"
"Me gusta esta película"
```

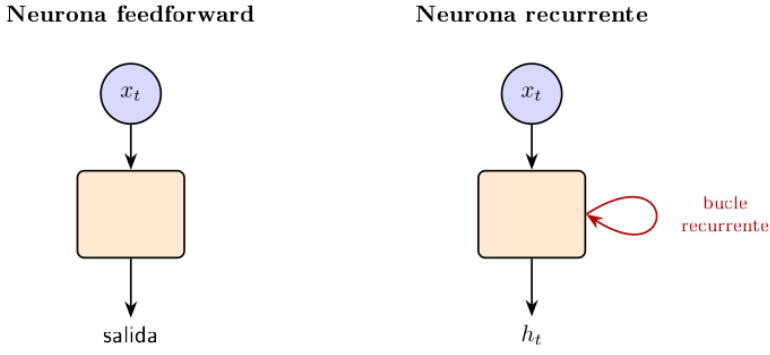
```
Los embeddings promediados son casi idénticos, ¡pero significan lo opuesto!
```

¿Por qué? El orden de las palabras importa. El contexto importa.

Solución: Usar modelos que procesen secuencias paso a paso

¿Qué es una Neurona Recurrente?

Una neurona ***feedforward*** (Clase 1) solo mira la entrada actual. Una **neurona recurrente** además mira su **propia salida del paso anterior**:



- La conexión que vuelve sobre sí misma es el **bucle recurrente**
- Permite que la neurona tenga **memoria**: lo que calculó antes influye en lo que calcula ahora
- Es la diferencia entre “ver una foto” y “ver una película”

Lectura: Goodfellow, Bengio & Courville (2016), “Deep Learning”, Cap. 10 — disponible gratis en `deeplearningbook.org`

El Mecanismo Paso a Paso

En cada instante de tiempo t, la neurona combina **dos fuentes de información**:

Fuente	Símbolo	Significado
Entrada actual	x_t	La palabra que estamos leyendo ahora
Estado anterior	h_{t-1}	El resumen de todo lo leído antes

Las tres operaciones de la neurona recurrente:

1. **Combinar**: mezcla la entrada nueva x_t con la memoria previa h_{t-1}

$$z_t = W_x \cdot x_t + W_h \cdot h_{t-1} + b$$

2. **Activar**: aplica una no linealidad (típicamente tanh)

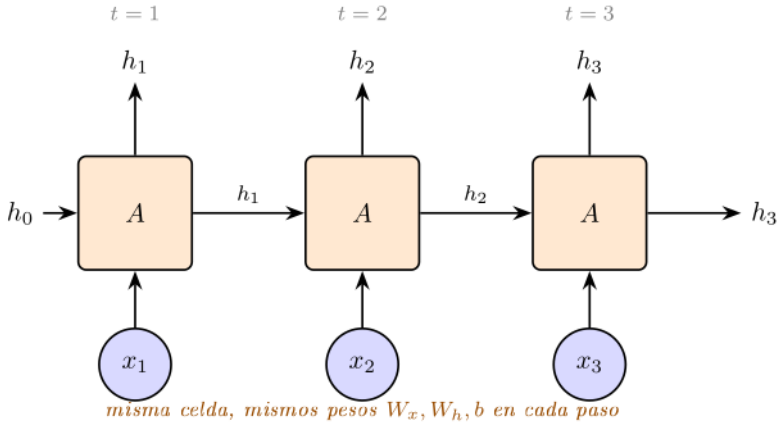
$$h_t = \tanh(z_t)$$

3. **Propagar**: h_t se convierte en la memoria para el paso $t + 1$

El artículo fundacional es Elman (1990), “Finding Structure in Time”, Cognitive Science 14(2). Introduce la RNN “simple” o “de Elman”.

Desplegando la Recurrencia en el Tiempo

El bucle es difícil de visualizar. El truco es **“desplegarlo”** (***unrolling***) y dibujar una copia de la neurona por cada paso temporal.



- **Clave**: la celda A es **la misma** en cada paso → mismos pesos W_x, W_h, b
- Esto se llama **compartición de parámetros** (***parameter sharing***)
- Permite procesar secuencias de **cualquier longitud** con un número fijo de parámetros

Analogía: como leer un libro con un solo cerebro. No usas un cerebro distinto por página; el mismo cerebro actualiza su comprensión página a página.

Visualización clásica: Olah (2015), “Understanding LSTM Networks”, blog `colah.github.io`. Para un tratamiento intuitivo, Karpathy (2015), “The Unreasonable Effectiveness of Recurrent Neural Networks”.

La Idea Central

Una RNN mantiene un **estado oculto** que evoluciona mientras lee palabras:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

```
"Me encantan los perros"


h_0 (inicio) → h_1 (tras "Me") → h_2 (tras "encantan") → h_3 (tras "perros")
               ↓                 ↓                      ↓
               salida_1          salida_2               salida_3
```

- **Estado oculto** h_t: resume “lo que sabemos hasta ahora”
- **Nueva palabra** x_t: actualiza el estado oculto
- **Recurrencia**: mismos pesos en cada paso (compartición de parámetros)

RNN vs. Red Feedforward

- **Feedforward** (Clase 1): procesa todas las entradas independientemente
- **RNN**: procesa entradas secuencialmente, el estado lleva información hacia adelante

¿Por qué las RNN tienen dificultades con secuencias largas?

Al retropropagar por muchos pasos temporales, los gradientes se reducen:

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_T} \cdot \prod_{t=1}^{T} \frac{\partial h_t}{\partial h_{t-1}}$$

Si cada derivada es < 1, el producto se vuelve diminuto (se desvanece)

Consecuencias: - Los estados iniciales no se actualizan correctamente - No se pueden aprender dependencias a largo plazo - “Fui a la escuela ayer. … La película fue excelente” (olvidó contexto)

Solución: Celdas LSTM (compuertas para controlar el flujo de información)

La Celda LSTM

Las LSTM añaden **compuertas** para controlar qué información mantener/descartar:

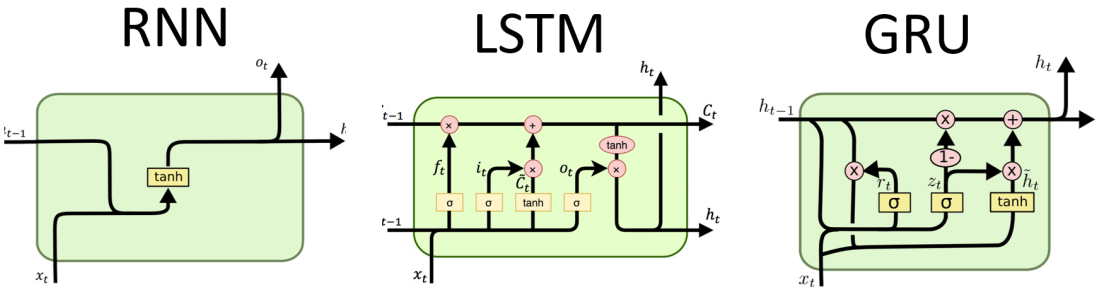
```
[Entrada] → [Compuerta de olvido] → [Compuerta de entrada] → [Estado celular] →
                                  ↓ ↓ ↓                 actualizaciones     ↓
                                 Actualizar            c_t                h_t
```

Tres Compuertas (Intuitivo)

Compuerta	Rol	Pregunta
Olvido	Borra información antigua	“¿Debo olvidar el contexto anterior?”
Entrada	Acepta nueva información	“¿Es importante la nueva palabra?”
Salida	Selecciona la salida	“¿Qué debo sacar ahora?”

Beneficio clave: Los gradientes fluyen mejor a través de compuertas (aditivos, no multiplicativos)

LSTM y GRU



Comparación de Código

```python
# RNN Simple
model = keras.Sequential([
    keras.layers.Embedding(vocab_size, embedding_dim),
    keras.layers.SimpleRNN(64, return_sequences=False),
    keras.layers.Dense(1, activation='sigmoid')
])

# LSTM (mejor)
model = keras.Sequential([
    keras.layers.Embedding(vocab_size, embedding_dim),
    keras.layers.LSTM(64, return_sequences=False),
    keras.layers.Dense(1, activation='sigmoid')
])
```

`return_sequences`:

- `True`: retorna estado oculto en cada paso temporal (para apilar)
- `False`: retorna solo el estado oculto final (para clasificación)

CNN para NLP: ¿Por qué?

Las CNN se diseñaron para imágenes pero funcionan bien para texto:

Idea: Usar filtros para detectar patrones de n-gramas

```
"La película fue absolutamente brillante"

Filtro (tamaño 2): detecta bigramas
"La película" → combinación positiva
"fue absolutamente" → sentimiento fuerte
```

- El filtro convolucional se desliza sobre secuencias de palabras
- Cada filtro aprende diferentes patrones
- Múltiples filtros capturan diferentes características (n-gramas)

CNN vs. RNN para Texto

Aspecto	RNN/LSTM	CNN
Largo alcance	(ok) Bueno (mediante compuertas)	(!) Depende del tamaño de filtro
Velocidad	(!) Secuencial (lento)	(ok) Paralelo (rápido)
Detección n-grama	(ok) Implícita	(ok) Explícita
Eficiencia param.	(ok) Menos parámetros	(!) Muchos filtros

Tarea de Análisis de Sentimiento (Revisitada)

Objetivo: Clasificar reseñas de IMDb como positivas/negativas

Datos: 25.000 entrenamiento, 25.000 prueba

Tres enfoques para comparar:

1. Embeddings promediados + Dense (realizado en Clase anterior)
2. LSTM (visto en sesión actual)
3. CNN (a realizar por c/estudiante, integrando contenidos ya vistos)

Relleno y Enmascaramiento

Problema: Las reseñas tienen diferentes longitudes (50-2000 palabras)

Solución 1: Relleno

```
x_padded = pad_sequences(x, maxlen=200, padding='post')
# Reseñas cortas: relleno con ceros al final
# Reseñas largas: truncar a 200 palabras
```

Solución 2: Enmascaramiento

```
keras.layers.Masking(mask_value=0.0)
# Decirle a la red: ignora los ceros rellenos en el cálculo
```

Compensación:

- Longitud fija: simple, pero pierde información
- Relleno: añade ruido (ceros)
- Enmascaramiento: dice al modelo ignorar el relleno

Contexto de Ambas Direcciones

LSTM estándar lee de izquierda a derecha:

```
"La película fue muy buena" → estados ocultos hacia adelante
```

LSTM Bidireccional: Lee en ambas direcciones

```
Adelante:  La → película → fue → muy → buena
                                         ↓
Atrás:     buena ← muy ← fue ← película ← La
```

Concatenar adelante + atrás ⟶ contexto más rico

```
keras.layers.Bidirectional(
    keras.layers.LSTM(64)
)
```

Múltiples Capas LSTM

Los modelos más profundos aprenden patrones más abstractos:

```
model = keras.Sequential([
    keras.layers.Embedding(5000, 128),
    keras.layers.Bidirectional(
        keras.layers.LSTM(64, return_sequences=True)
    ),
    keras.layers.Bidirectional(
        keras.layers.LSTM(32, return_sequences=False)
    ),
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
```

Nota: Con `return_sequences=True` se pasan todos los pasos temporales a la siguiente capa

Ideas clave

- Las RNN procesan secuencias paso a paso
- Las LSTM resuelven el problema de gradientes desvanecientes con compuertas de olvido
- Uso de relleno y enmascaramiento para longitudes variables
- Procesamiento bidireccional captura contexto en ambas direcciones
- Apilar capas aumenta la capacidad

Solución de Problemas

Problema	Causa	Solución
Entrenamiento lento	Naturaleza secuencial de RNN	Usar CNN o Transformers (próxima clase)
Malos resultados LSTM	Gradientes desvanecientes persisten	Añadir dropout, regularización
Sobreajuste	Demasiados parámetros	Reducir tamaño LSTM, añadir Dropout
Error de memoria	Secuencias largas	Reducir max_length, usar procesamiento por lotes

Referencias Completas

Para que los estudiantes profundicen por su cuenta:

- **Goodfellow, I., Bengio, Y. & Courville, A. (2016).** ***Deep Learning***. MIT Press. Capítulo 10: "Sequence Modeling: Recurrent and Recursive Nets". Libro de texto estándar, gratuito en `https://www.deeplearningbook.org`.
- **Elman, J. L. (1990).** "Finding Structure in Time". ***Cognitive Science***, 14(2), 179–211. Artículo fundacional que introduce la RNN simple (o "de Elman").
- **Olah, C. (2015).** "Understanding LSTM Networks". Blog ***colah's blog***: `https://colah.github.io/posts/2015-08-Understanding-LSTMs/`. La explicación visual más citada sobre RNN y LSTM.

- **Karpathy, A. (2015).** “The Unreasonable Effectiveness of Recurrent Neural Networks”. `https://karpathy.github.io/2015/05/21/rnn-effectiveness/`. Complemento motivacional con ejemplos sorprendentes de generación de texto.

- **Werbos, P. J. (1990).** “Backpropagation Through Time: What It Does and How to Do It”. ***Proceedings of the IEEE***, 78(10), 1550–1560. Para quienes quieran el detalle del algoritmo de entrenamiento (conecta con la lámina de gradientes desvanecientes).